

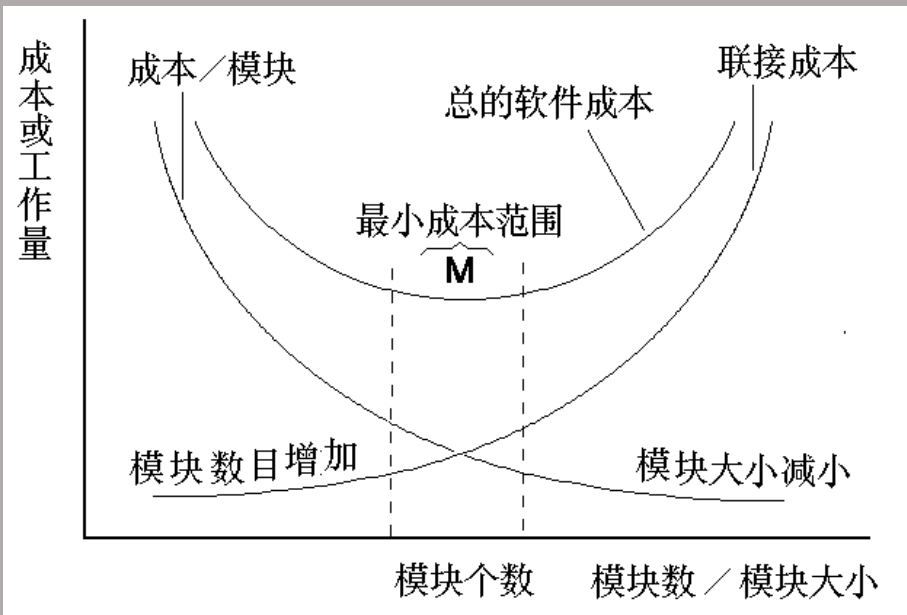


软件的模块化

- 模块(module)定义：整个软件可被划分成若干个可单独命名且可编址组成部分，这些部分称之为模块。
- 模块具有如下三个基本属性：
 - 功能：实现什么功能，做什么事情。
 - 逻辑：描述模块内部怎么做。
 - 状态：该模块使用时的环境和条件。
- 模块的表示
 - 模块的外部特性：是指模块的模块名、参数表、以及给程序以至整个系统造成的影响。
 - 模块的内部特性：是指完成其功能的程序代码和仅供该模块内部使用的数据。

软件模块的划分

- 按照自顶向下的设计原则，需将一个大规模的软件分解成若干相对独立的模块，然后分别对这些规模较小的模块进行处理。
- 难点在于合理的划分模块
 - 如果模块是相互独立的（模块独立性），当模块变得越小，每个模块花费的工作量越低
 - 但当模块数增加时，模块间的联系也随之增加（模块的耦合度），把这些模块联接起来的工作量也随之增加。



降低了系统的复杂性，使得系统容易修改；
推动了系统各个部分的并行开发，从而提高了软件的生产效率。



模块设计标准

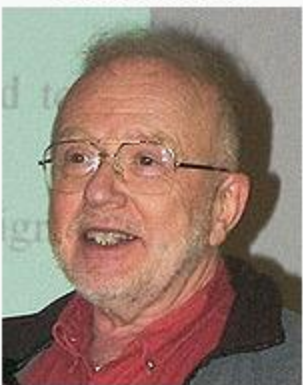
- 模块可分解性：可将系统按问题 / 子问题分解的原则分解成系统的模块层次结构;
- 模块可组装性：可利用已有的设计构件组装成新系统，不必一切从头开始。
- 模块可理解性：一个模块可不参考其他模块而被理解；
- 模块连续性：对软件需求的一些微小变更只导致对某个模块的修改而整个系统不用大动;
- 模块保护：将模块内出现异常情况的影响范围限制在模块



信息隐藏

- **David Parnas** 指出，每个模块的实现细节对于其它模块来说应该是隐蔽的。模块中所包含的信息（包括数据和过程）不允许其它不需要这些信息的模块使用。
- 其最大的好处就是使得模块的修改和软件的维护所造成的影响可以局限在一个或几个模块范围内。
- 模块之间的关系能够达到信息隐藏，就可以认为是合理的模块划分。

David Parnas



David Lorge Parnas (born February 10, 1941) is a [Canadian](#) early pioneer of [software engineering](#), who developed the concept of [information hiding in modular programming](#), which is an important element of [object-oriented programming](#) today. He is also noted for his advocacy of precise documentation.



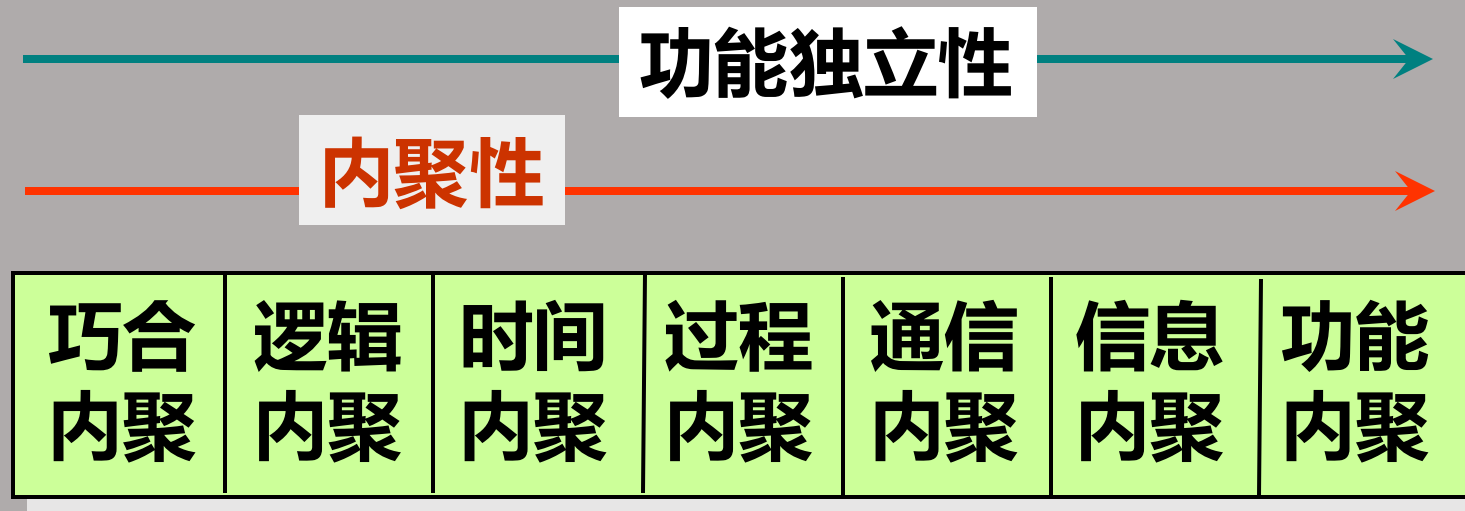
模块的独立性

- 功能独立性是抽象、模块化和信息隐藏的直接产物。
- 如果一个模块能够独立于其他模块被编程、测试和修改，而和软件系统中其它的模块的接口是简单的，则该模块具有功能独立性。
- 1978年Meyer提出了两个准则度量模块独立性，即模块间的耦合和模块的内聚。



模块的内聚性

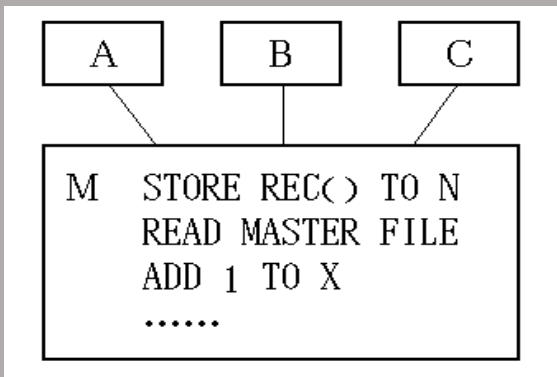
- 内聚是模块功能强度的度量，一个模块内部各元素之间的联系越紧密，则它的内聚性就越高，相对地，它与其他模块之间的耦合性就会减低，而模块独立性就越强。





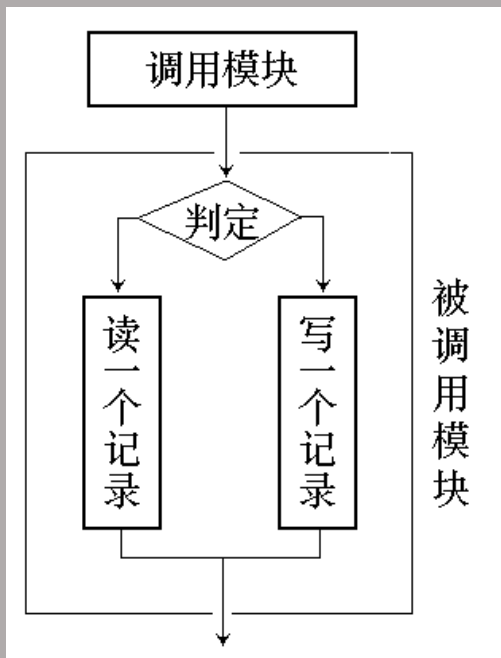
- 巧合内聚

当几个模块内凑巧有一些程序段代码相同，又没有明确表现出独立的功能，为了减少存储把这些代码独立出来建立一个新的模块，这个模块就是巧合内聚模块。它是内聚程度最低的模块。



- 逻辑内聚

这种模块把几种相关的功能组合在一起，每次被调用时，由传送给模块的判定参数来确定该模块应执行哪一种功能。





模块的内聚性

- 时间内聚

- 时间内聚又称为经典内聚。这种模块一般为多功能模块，但模块的各个功能的执行与时间有关，通常要求所有功能必须在同一时间段内执行。
- 例如初始化模块和终止模块。

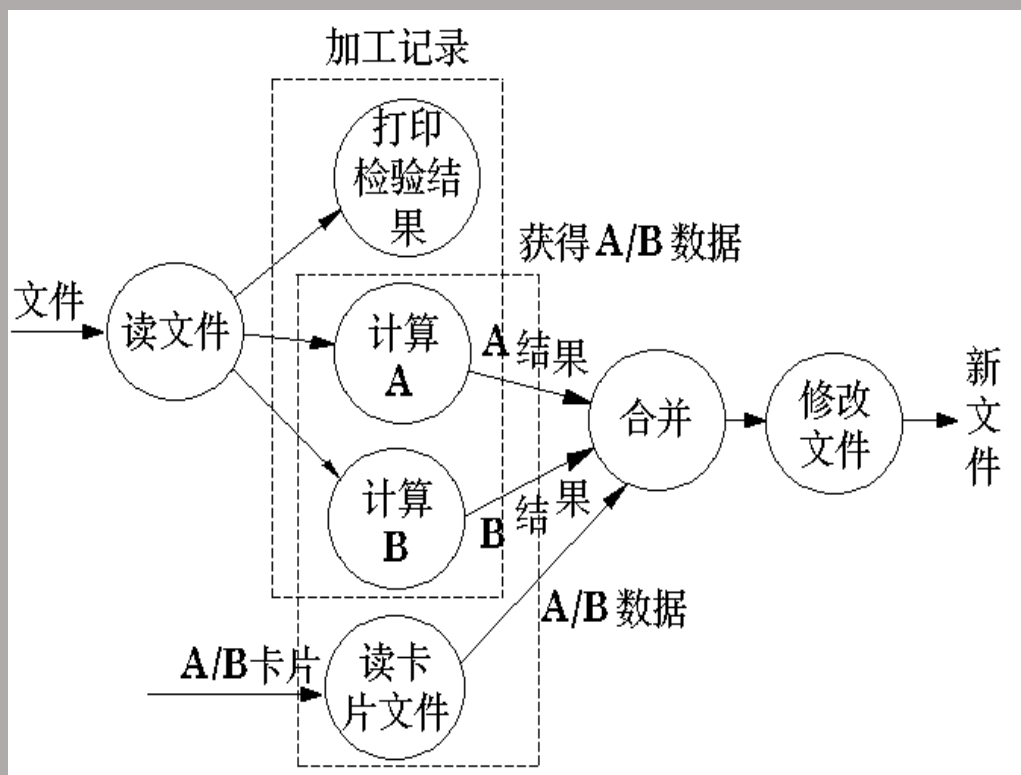
- 过程内聚

- 使用流程图做为工具设计程序时，把流程图中的某一部分划出组成模块，就得到过程内聚模块。
- 例如，把流程图中的循环部分、判定部分、计算部分分成三个模块，这三个模块就是过程内聚模块。



- 通信内聚

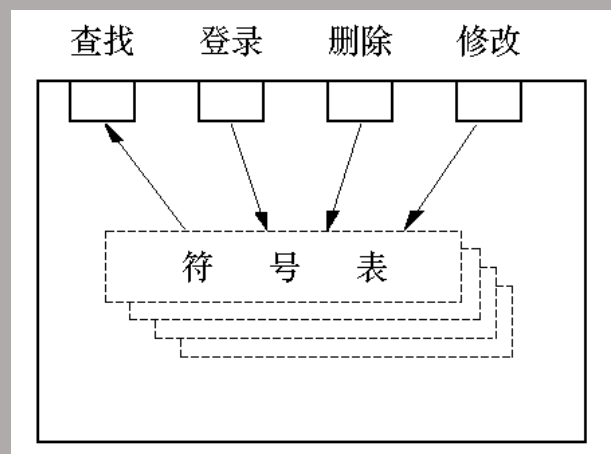
- 如果一个模块内各功能部分都使用了相同的输入数据，或产生了相同的输出数据，则称之为通信内聚模块。通常，通信内聚模块可通过数据流图来判定。





- 信息内聚

- 这种模块完成多个功能，各功能都在同一数据结构上操作，每一项功能有一个唯一的入口点。
- 这个模块将根据不同的要求，确定该执行哪一个功能。由于这个模块的所有功能都是基于同一个数据结构（符号表），因此，它是一个信息内聚的模块。
- 信息内聚模块可以看成是多个功能内聚模块的组合，并且达到信息的隐蔽。



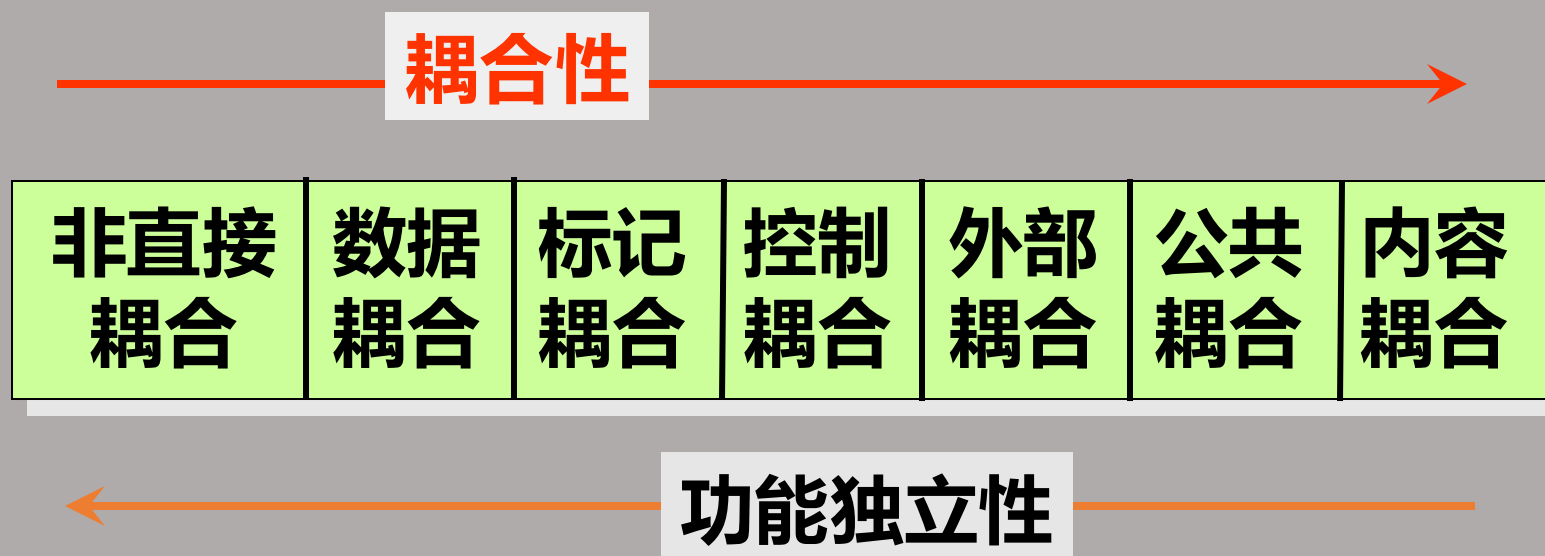
- 功能内聚

- 一个模块中各个部分都是完成某一具体功能必不可少的组成部分，或者说该模块中所有部分都是为了完成一项具体功能而协同工作，紧密联系，不可分割的。则称该模块为功能内聚模块。



模块的耦合性

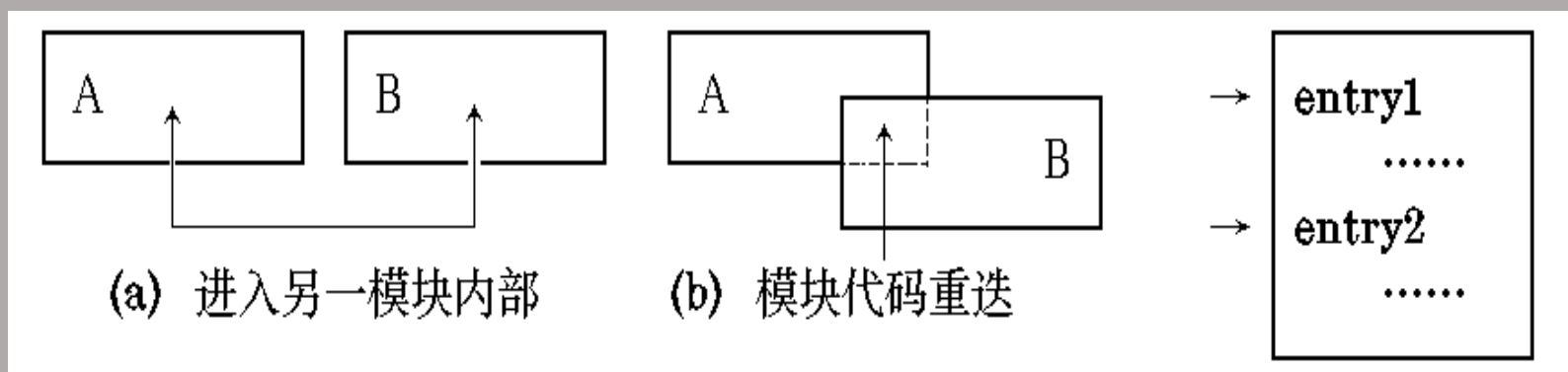
- 耦合是模块之间互相连接的紧密程度的度量。模块之间的连接越紧密，联系越多，耦合性就越高，而其模块独立性就越弱。





- 内容耦合

- 一个模块直接访问另一个模块的内部数据;
- 一个模块不通过正常入口转到另一模块内部;
- 两个模块有一部分程序代码重迭(汇编语言中);
- 一个模块有多个入口。

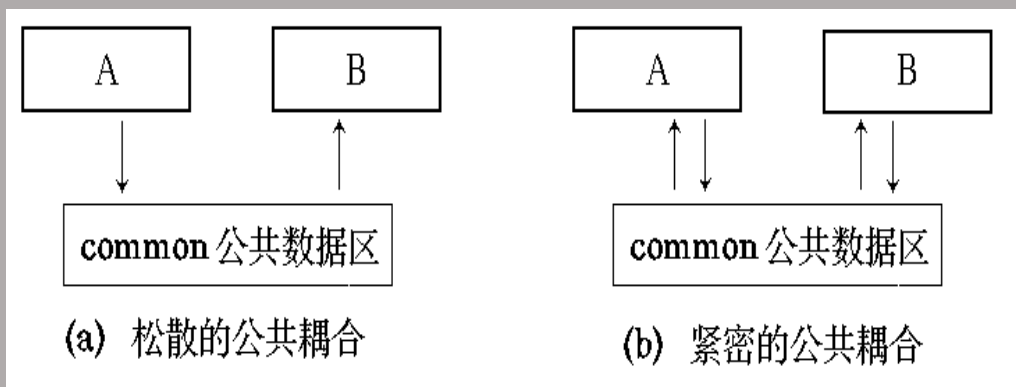




- 公共耦合 (Common Coupling)

若一组模块都访问同一个公共数据环境，则它们之间的耦合就称为公共耦合。公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等。

公共耦合的复杂程度随耦合模块的个数增加而显著增加。若只是两模块间有公共数据环境，则公共耦合有两种情况。松散公共耦合和紧密公共耦合。



公共耦合会引起下列问题：

所有公共耦合模块都与某一个公共数据环境内部各项的物理安排有关，若修改某个数据的大小，将会影响到所有的模块。

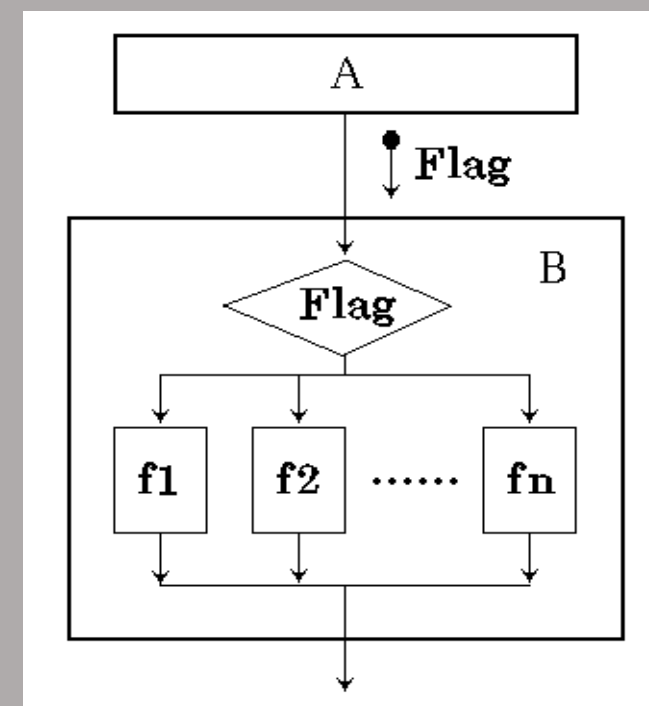
无法控制各个模块对公共数据的存取，严重影响软件模块的可靠性和适应性。

公共数据名的使用，明显降低了程序的可读性。



模块的耦合性

- 外部耦合
一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息，则称之为外部耦合。
- 控制耦合
如果一个模块通过传送开关、标志、名字等控制信息，明显地控制选择另一模块的功能，就是控制耦合。
 - 控制耦合是在单一接口上选择多功能某块中的某项功能。
 - 对所控制模块的任何修改，都会影响控制模块
 - 且控制模块必须知道所控制模块内部的一些逻辑关系，降低了模块的独立性。





- 标记耦合

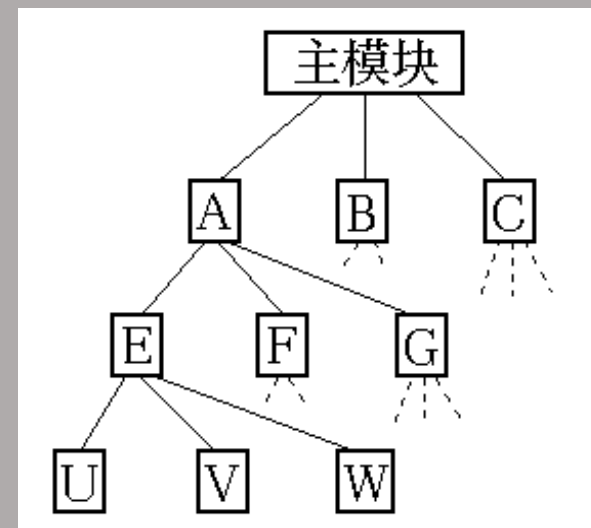
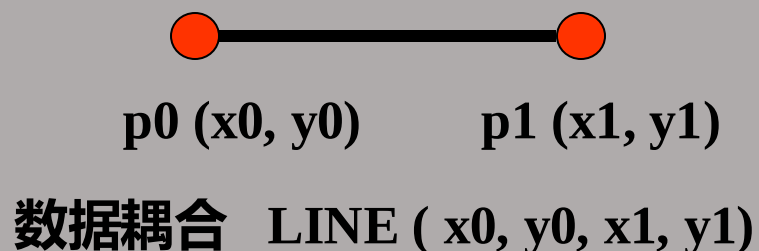
- 如果一组模块通过参数表传递记录信息，就是标记耦合。这个记录是某一数据结构的子结构，而不是简单变量。
- 要求这一组模块都必须清楚该数据结构，并按结构的要求进行操作。它使在数据结构上的操作复杂化了，应该把数据结构上的操作全部集中在一个模块中，来消除这种耦合。



标记耦合 LINE (p0, p1)

• 数据耦合

- 如果一个模块访问另一个模块时，彼此之间是通过简单数据参数（不是控制参数、公共数据结构或外部变量）来交换输入、输出信息的，则称这种耦合为数据耦合。
- 由于限制了只通过参数传递数据，该方式开发的程序简单、安全可靠，因此它是一种松散的耦合，模块间独立性强。



• 非直接耦合

- 如果两个模块之间没有直接关系，它们之间的联系完全是通过主（上级）模块的控制和调用来实现的，这就是非直接耦合。这种耦合的模块独立性最强。



模块的耦合度计算

- $$\text{Coupling}(C) = 1 - \frac{1}{d_i + 2 \times c_i + d_o + 2 \times c_o + g_d + 2 \times g_c + w + r}$$

- 其中针对数据参数和控制参数的耦合项:

- d_i : 数据参数的输入个数; c_i : 控制参数的输入个数;
- d_o : 数据参数的输出个数; c_o : 控制参数的输出个数;

- 针对全局耦合项:

- g_d : 作为数据的全局变量个数;
- g_c : 作为控制的全局变量个数;

- 针对环境耦合项:

- w : 调用的模块数, 也称为扇出数;
- r : 调用的模块数, 也称为扇入数。

Coupling(c)取值越大意味着模块之间的耦合度越大。根据经验, 该公式的取值范围应该大约在0.67 (低耦合) 到1.0 (高耦合)



降低模块耦合度的方法

- 根据问题的特点选择适当的耦合类型
 - 模块间的信息传递：数据信息和控制信息的选择；
 - 模块间的调用方式：传送地址和传送判定参数的选择；
 - 系统的错误处理模块：集中处理与分散处理的选择；
- 降低模块接口的复杂度
 - 传送信息的数量：参数多和参数少的区别；
 - 模块的调用方式：简单调用和直接引用的区别；
- 将模块的通信信息放在缓冲区中



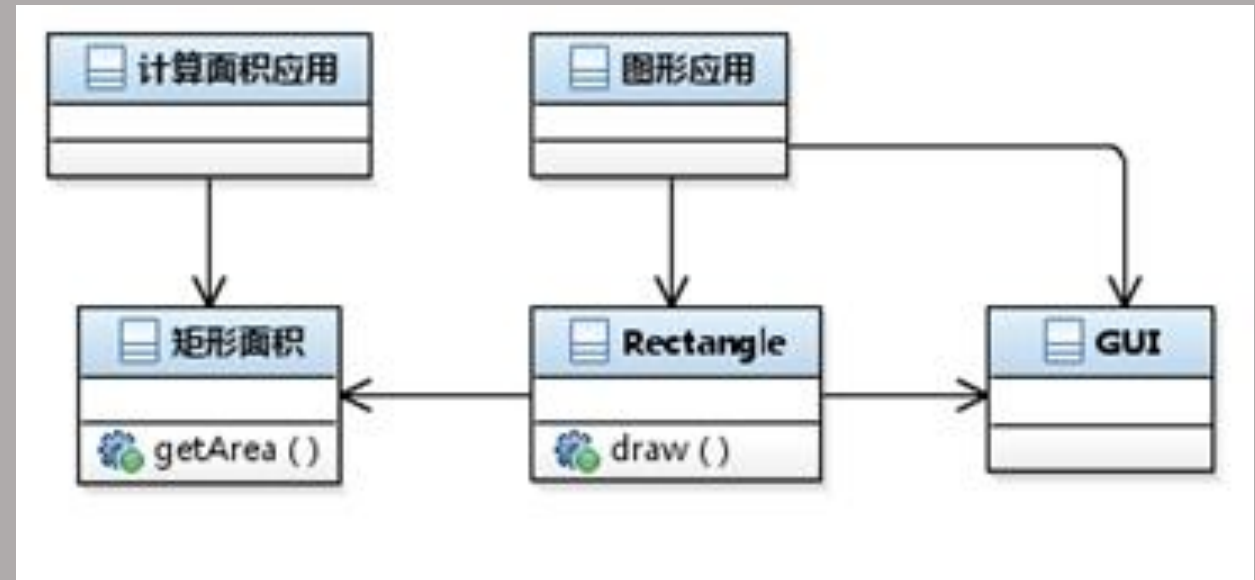
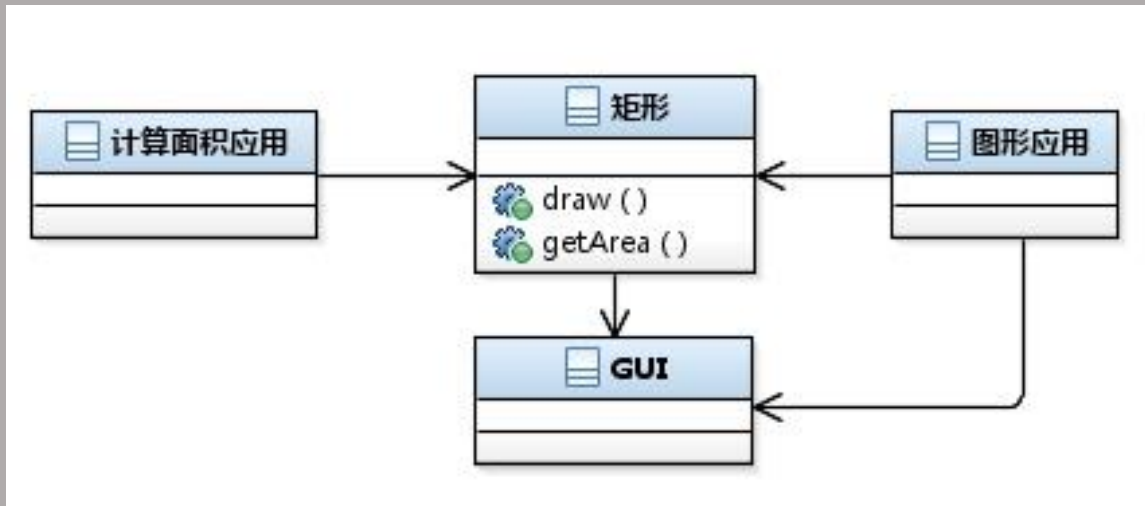
面向对象的设计原则

- 单一职责 (Single Responsibility)
- 里氏替换原则 (Liskov Substitution)
- 依赖倒置原则 (Dependency Inversion)
- 接口隔离原则 (Interface Segregation)
- 迪米特法则 (Law of Demeter)
- 开闭原则 (Open Closed)
- 组合/聚合复用 (Composite/Aggregation Reuse)



单一职责 (Single Responsibility)

- 定义：针对类，应该只有一个引起它变化的原因，“职责”定义为变化的原因。
 - 如果有其他原因去改变一个类，那么这个类就具有其他的职责。类具有多个职责，等于这些职责具有耦合关系。
 - 为了提高类的内聚度，应将对象的不同职责分离至两个或多个类中，确保引起该类变化的原因只有一个。





里氏替换原则 (Liskov Substitution)

- 子类应当可以替换父类并出现在父类能够出现的任何地方。
 - 原始定义如下：若对于每一个类型S的对象o1，都存在一个类型T的对象o2，使得在所有针对T编写的程序P中，用o1替换o2后，程序P的行为功能不变，则S是T的子类型。
 - LSP告诉我们：如果一个软件对象是父类型的，那么它出现的所有地方都可以被子类型对象替换。所有子类的行为功能必须和客户类对其父类所期望的行为功能保持一致。
 - OCP原则强调对变化的类进行抽象，LSP则强调了实现抽象化的具体规范。





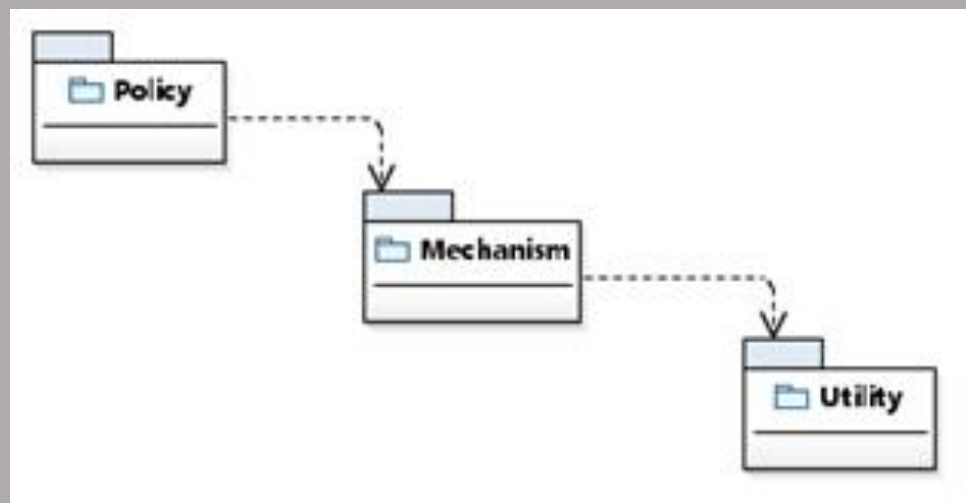
依赖倒置原则 (Dependency Inversion)

- 高层模块不应依赖于低层模块，二者都应依赖于抽象；抽象不应依赖于实现细节，细节应该依赖于抽象。然而，传统的模块间的调用关系与该原则正好相反，低层模块的改动使得高层模块不得不随之改动。
 - 高层模块包含应用程序的策略规则和业务逻辑，低层模块是具体的实现细节。
 - 低层模块的改动不应影响高层的逻辑；
 - 软件设计能够重用高层模块；
- 程序中所有的依赖关系应该终止于抽象类或者接口：
 - 任何变量都不应该持有一个指向具体类的指针或者引用；
 - 任何类都不应该从具体类派生，或者说继承自一个具体类；
 - 任何方法都不应该覆盖它的任何基类中的已经实现了的方法。

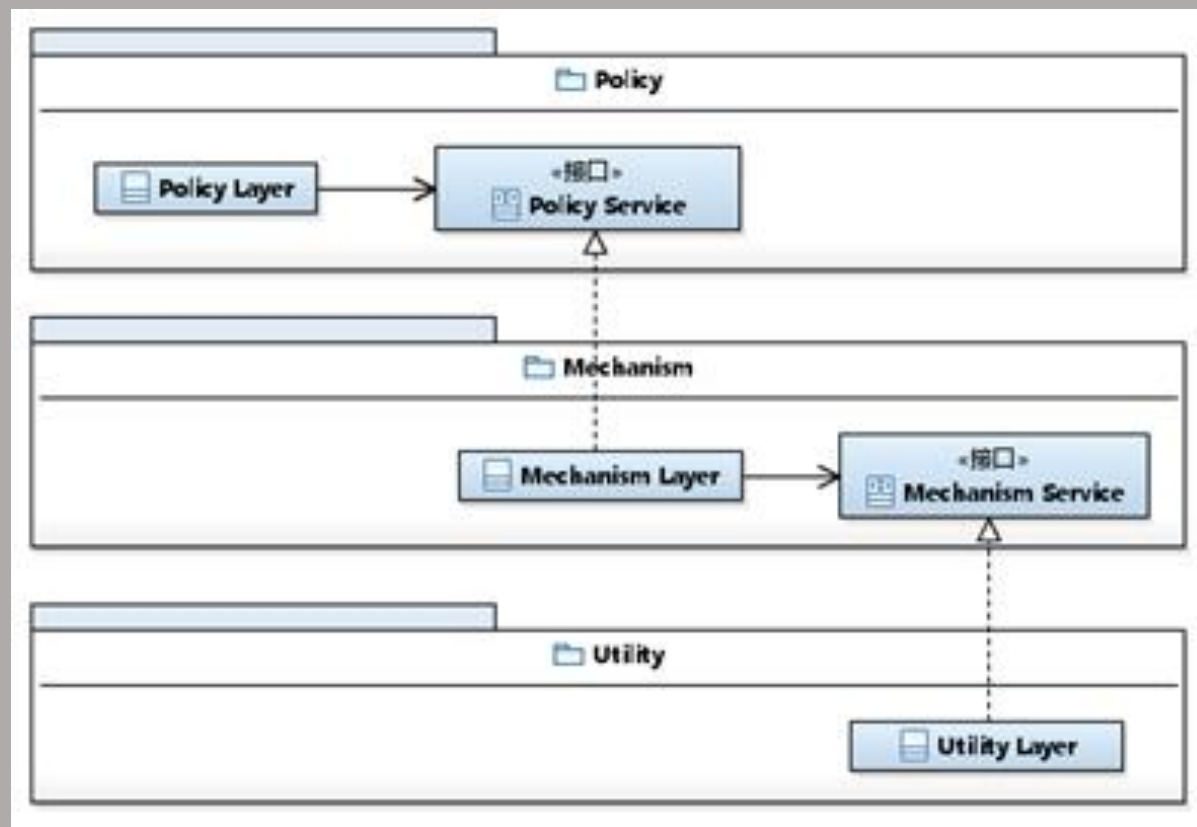


依赖倒置原则（Dependency Inversion）

“...所有结构良好的面向对象架构都具有清晰的层次定义，每个层次通过一个定义良好的、受控的接口向外提供了一组内聚的服务”。对这个陈述的简单理解可能导致设计者设计出类似下图的结构分层包图。



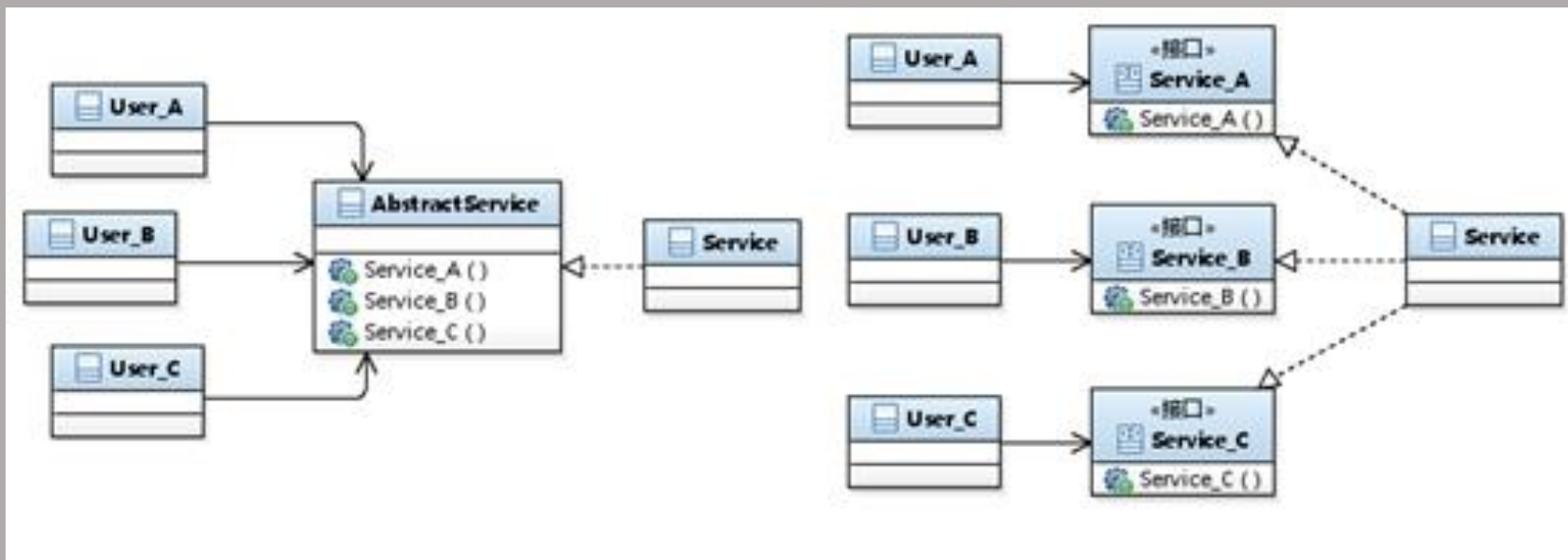
每个较高层次都为它所需要的服务声明一个抽象接口，较低的层次实现了这些抽象接口，每个高层类都通过该抽象接口使用下一层的服务。





接口隔离原则 (Interface Segregation)

- 采用多个与特定客户类的接口 比 采用一个通用的涵盖多个业务方法的接口要好。
- 如果一个服务器类为多个客户类提供不同的服务，则服务器类应该为每一个客户类创建特定的业务接口，而不要为所有客户类提供统一的业务接口，除非这些客户类请求的服务相同。





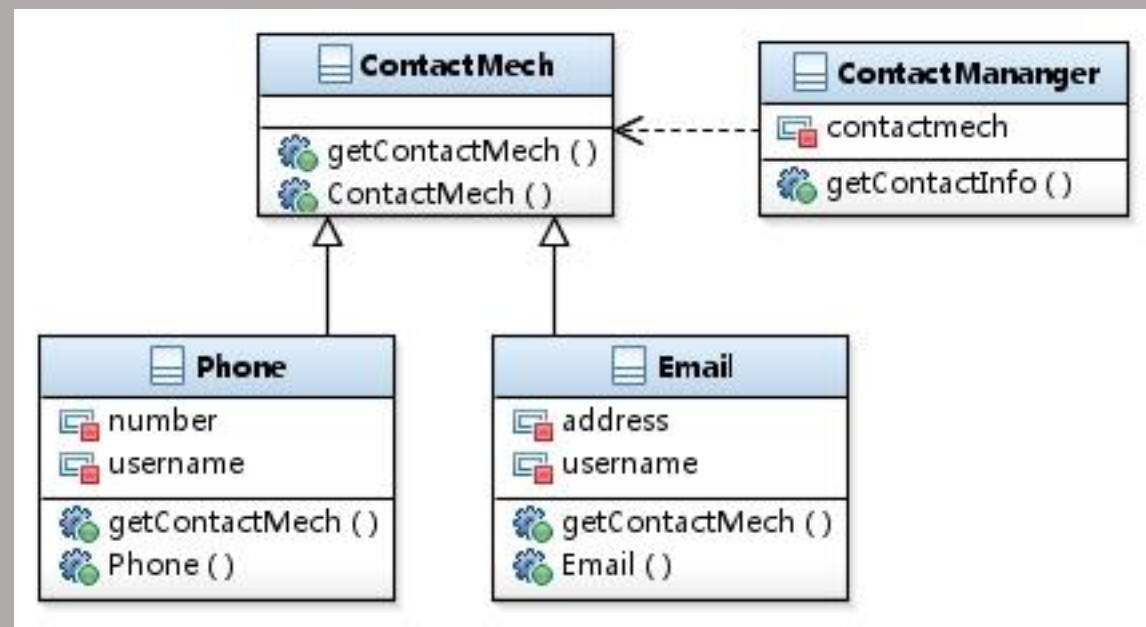
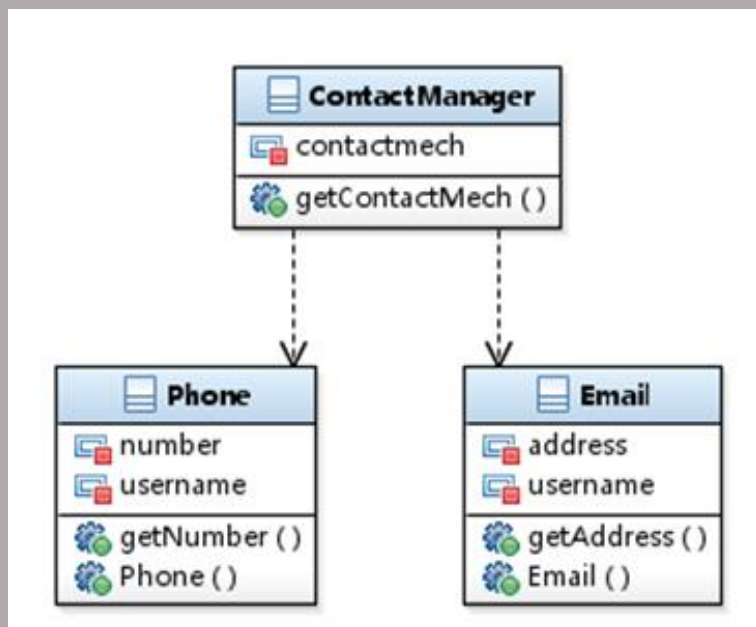
迪米特法则 (Law of Demeter)

- 最少知识原则：一个对象应当可能少的了解其它对象。
- 只与你直接的朋友们通信，不要跟“陌生人”说话。
- 符合以下特征的属于朋友：
 - 当前对象本身 (this) ；
 - 以参量形式传入到当前对象方法中的对象；
 - 当前对象的实例变量直接引用的对象；
 - 当前对象的实例变量如果是一个聚集，那么聚集中的元素也都是朋友；
 - 当前对象所创建的对象。



开闭原则 (Open-Closed)

- 软件实体（类、模块、函数）可以扩展，但不能修改。
 - 对于扩展是开放的（Open 4 extension），需求改变时，对实体进行扩展。
 - 对于更改是封闭的（Closed 4 modification），需求改变时，禁止对原来的代码进行修改。
 - 实现OCP的关键是使用抽象，识别不同类之间的共性和变化点，利用封装对变化点进行处理。





组合/聚合复用原则 (Composite/Aggregation Reuse)

- 在一个新对象里面使用一些已有对象，使之成为新对象的一部分；新对象通过向已有对象委托 (delegate) 一部分责任而达到复用已有对象的目的。
- 两种复用的方式：

继承复用

优点：简单易用

缺点：

- 1、破坏封装性；
- 2、LSP原则得不到保障；
- 3、父类更改导致子类必须更改；
- 4、继承的实现为静态，缺乏灵活性
- 5、依赖于相同的上下文环境；

组合/聚合复用

将已有对象组合/聚合为一个新对象的方式实现对已有对象行为功能的复用

优点：

- 1、支持封装性，实现黑盒复用；
- 2、新对象符合SRP原则；
- 3、通过接口实现动态引用；
- 4、不依赖于上下文；

缺点：

- 1、将已有对象扩充到新对象中比较困难；
- 2、会产生大量新对象，管理困难；



软件设计基础

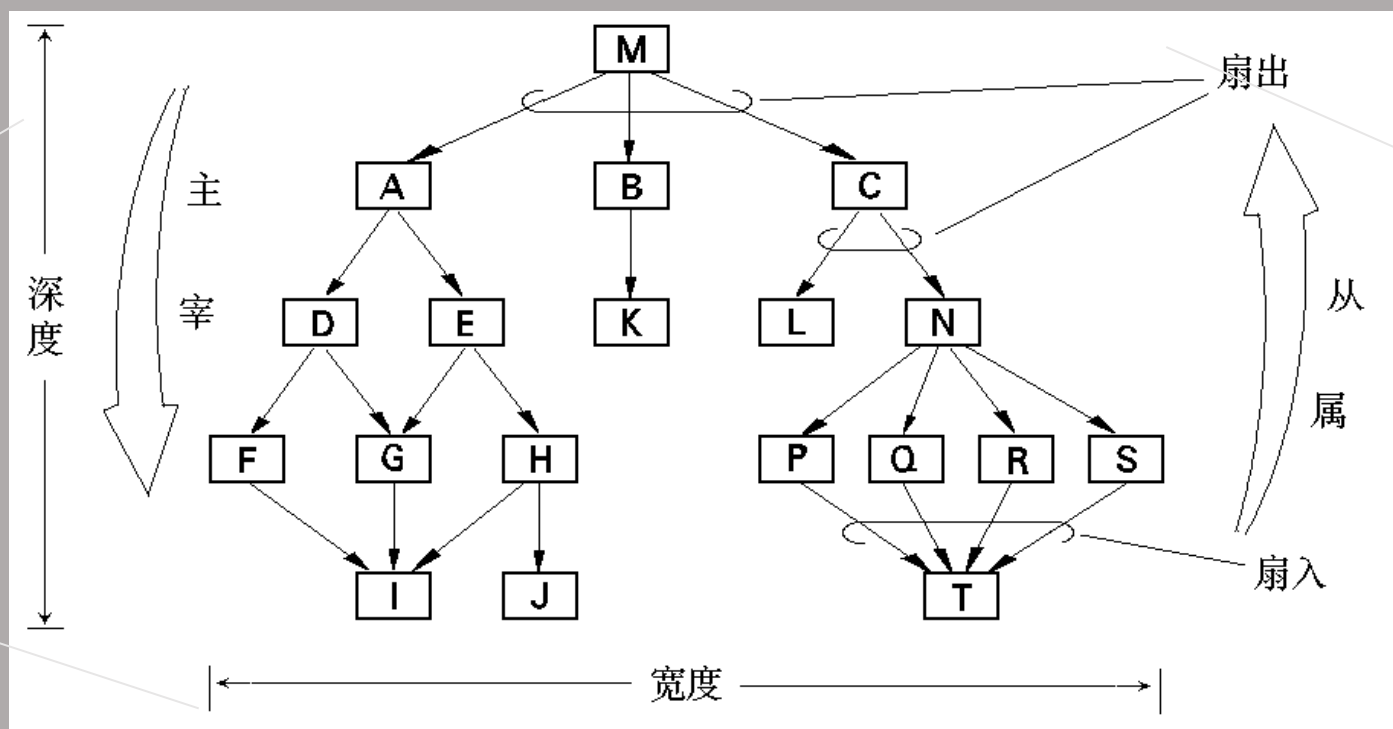
- 自顶向下，逐步细化
- 系统控制结构
- 结构划分和结构图
- 数据结构
- 软件过程



自顶向下逐步细化

- 这是Niklaus Wirth提出的设计策略。
- 将软件的体系结构按自顶向下方式，对各个层次的过程细节和数据细节逐层细化，直到用程序设计语言的语句能够实现为止，从而最后确立整个的体系结构。
 - 最初的说明只是概念性地描述了系统的功能或信息，但并未提供有关功能的内部实现机制或有关信息的内部结构的任何信息。
 - 对初始说明仔细推敲，进行功能细化或信息细化，给出实现的细节，划分出若干成份。

- 系统控制结构表明了程序构件（模块）的组织情况。控制层次往往用程序的层次（树形或网状）结构来表示。



反映了程序结构的规模和复杂程度

反映了程序结构的控制规模

多扇出意味着需要控制和协调许多下属模块。而多扇入的模块通常是公用模块

- 软件过程就是软件的详细设计，是对确定的软件模块确定内部逻辑的活动。
- 软件过程必须提供精确的处理说明，包括事件的顺序、正确的判定点、重复的操作直至数据的组织和结构等等。
- 程序结构与软件过程是有关系的。对每个模块的处理必须指明该模块所在的上下级环境。
- 软件过程遵从程序结构的主从关系，因此它也是层次化的。

